

46. Une table gravée

Ce qui ne change jamais n'a rien à faire en mémoire.

1. Ce qu'il faut comprendre

`const uint8_t T[]` est **gravé dans la cartouche**, comme une tuile. Le compilateur l'annonce : ces octets sortent du décompte de la mémoire de travail.

Ôter le `const` la met en mémoire — et la rend modifiable, et la rend vide au démarrage. Les deux formes ont leur usage ; la différence se voit dans le compte rendu de compilation, et nulle part ailleurs.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1362 octets** de cartouche.

1. UNE TABLE GRAVÉE DANS LA CARTOUCHE — COLONNES

```
const uint8_t COLONNES[] = { 2, 5, 8, 11, 14, 17 };
```

2. UNE TABLE GRAVÉE DANS LA CARTOUCHE — LIGNES

```
const uint8_t LIGNES[] = { 3, 6, 9, 6, 3, 6 };
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

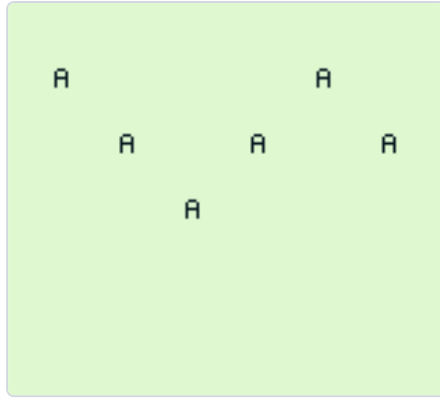
```
int main() {  
    for (uint8_t i = 0; i < sizeof(COLONNES); i++)  
        poser(COLONNES[i], LIGNES[i], 1);  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Six tuiles posées en zigzag, aux positions de la table.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la première est posée
- ✓ la quatrième aussi
- ✓ la dernière ferme le zigzag
- ✓ et rien n'est posé entre deux

Ensuite : **47. Une grille** — niveau 6 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++